

ЗВІТ
до лабораторної роботи № 8
на тему:
"Розширені можливості Redis."

Виконала:
студентка групи МІТ-31
Йовхимищ Діана

Мета роботи: закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams.

Хід роботи

Завдання:

1. Транзакції у Redis.

- а. Ознайомтесь з командами MULTI, EXEC, DISCARD, WATCH.
- б. Створіть просту транзакцію, що додає кілька ключів.**

Запустила Redis:

```
diana@DESKTOP-2RG0T4B:~$ redis-cli  
127.0.0.1:6379> |
```

Виконала транзакцію в Redis за допомогою команди MULTI.

У межах транзакції створила кілька ключів (user:1, age, city). Команди були поставлені в чергу (QUEUED) та виконані одночасно після виклику EXEC. Результат виконання підтверджено відповіддю OK для кожної операції:

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET user:1 "Diana"
QUEUED
127.0.0.1:6379(TX)> SET age 20
QUEUED
127.0.0.1:6379(TX)> SET city "Kyiv"
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) OK
3) OK
127.0.0.1:6379>
```

Перевіряємо результат:

```
127.0.0.1:6379> GET user:1
"Diana"
127.0.0.1:6379> GET age
"20"
127.0.0.1:6379> GET city
"Kyiv"
127.0.0.1:6379>
```

- с. Спробуйте використати WATCH для імітації конкурентного доступу.

Встановлюємо початкові значення:

```
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> SET purchases 0
OK
127.0.0.1:6379>
```

Далі використовуємо WATCH:

```
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECR balance
QUEUED
127.0.0.1:6379(TX)> INCR purchases
QUEUED
127.0.0.1:6379(TX)> EXEC
1) (integer) 99
2) (integer) 1
```

Змінюємо balance вручну:

```
127.0.0.1:6379> SET balance 999
OK
127.0.0.1:6379>
```

Перевіряємо виконану роботу:

```
127.0.0.1:6379> GET balance
"999"
127.0.0.1:6379> GET purchases
"1"
127.0.0.1:6379>
```

2. Lua-скрипти в Redis

- a. Ознайомтесь із командою EVAL.
- b. Напишіть простий Lua-скрипт, який перевіряє наявність ключа і створює його, якщо не існує:

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey "value"
OK
```

Перевіряємо значення ключа командою:

```
127.0.0.1:6379> GET testkey
"value"
127.0.0.1:6379>
```

3. Механізм Pub/Sub

- а. В одному терміналі запустіть підписку на канал:
<https://redis.io/docs/latest/commands/subscribe/>

```
127.0.0.1:6379> SUBSCRIBE news
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "news"
3) (integer) 1
```

- б. В іншому терміналі виконайте публікацію повідомлення:
<https://redis.io/docs/latest/commands/publish/>

```
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 1
127.0.0.1:6379>
```

```
127.0.0.1:6379> SUBSCRIBE news
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "news"
3) (integer) 1
1) "message"
2) "news"
3) "Redis is awesome!"
```

Було успішно реалізовано механізм Pub/Sub:

1. клієнт-підписник отримує повідомлення миттєво після публікації
2. передача даних відбувається в реальному часі

4. Redis Streams (поток даних)

<https://redis.io/docs/latest/develop/tools/insight/tutorials/insight-stream-consumer/>

a. Додайте події до потоку:

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
"1775046857597-0"
127.0.0.1:6379> XADD mystream * sensor-id 5678 temperature 21.5
"1775046867272-0"
127.0.0.1:6379> |
```

b. Прочитайте останні події:

```
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1775046857597-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
2) 1) "1775046867272-0"
   2) 1) "sensor-id"
      2) "5678"
      3) "temperature"
      4) "21.5"
127.0.0.1:6379>
```

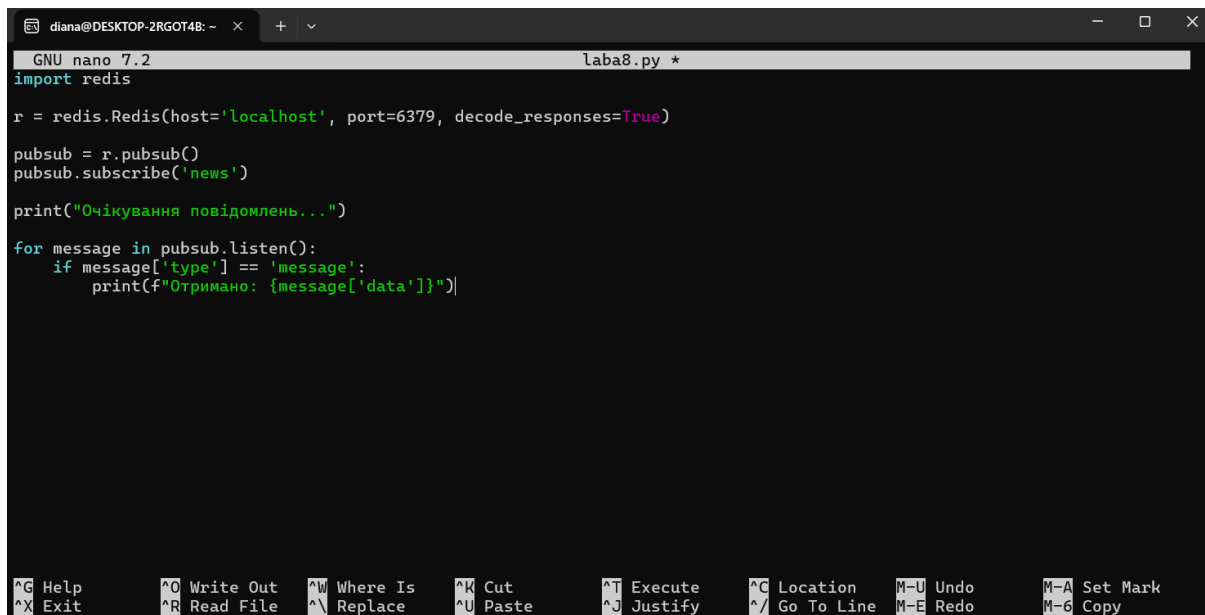
У результаті отримано два записи з даними сенсорів (ідентифікатор та температура).

c. Створіть споживача потоку та зчитайте нові повідомлення:

```
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
   2) 1) 1) "1775046857597-0"
      2) 1) "sensor-id"
         2) "1234"
         3) "temperature"
         4) "19.8"
      2) 1) "1775046867272-0"
         2) 1) "sensor-id"
            2) "5678"
            3) "temperature"
            4) "21.5"
```

Було успішно зчитано два повідомлення з потоку, починаючи з початку (0).

5. (Додатково) Напишіть скрипт (Python/JS), який реалізує логіку Pub/Sub або читає дані зі Stream.



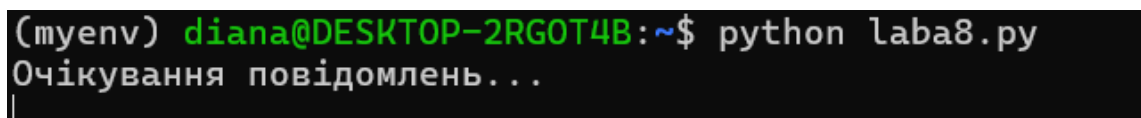
```
GNU nano 7.2      laba8.py *
import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)

pubsub = r.pubsub()
pubsub.subscribe('news')

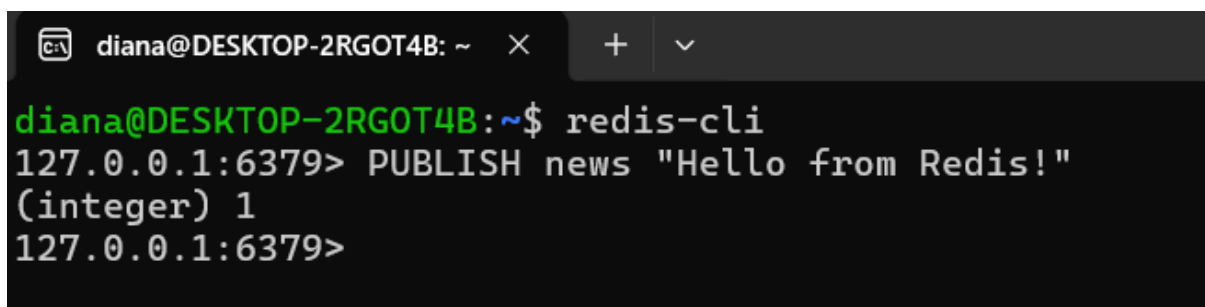
print("Очікування повідомлень...")

for message in pubsub.listen():
    if message['type'] == 'message':
        print(f"Отримано: {message['data']}")
```



```
(myenv) diana@DESKTOP-2RGOT4B:~$ python laba8.py
Очікування повідомлень...
```

Відправка повідомлення через Redis CLI в іншому терміналі:



```
diana@DESKTOP-2RGOT4B:~$ redis-cli
127.0.0.1:6379> PUBLISH news "Hello from Redis!"
(integer) 1
127.0.0.1:6379>
```

```
(myenv) diana@DESKTOP-2RGOT4B:~$ python laba8.py
Очікування повідомлень...
Отримано: Hello from Redis!
```

Скрипт успішно підписується на канал Redis і отримує повідомлення в реальному часі. Це демонструє механізм Pub/Sub, де підписник отримує повідомлення одразу після публікації.